

Hackathon Hack-Pack: The "AI Director's Cut"

Overview

Your challenge today is to build a local web application that takes a raw, unedited video file (e.g., a podcast, gameplay footage, or a home video), uses local/API-driven ML and GenAI to analyze the media, and automatically generates a **curated, stylized highlight reel** with automated smart cuts, background music matching, dynamic visual overlays, and AI voice narration.

The vibe: You are building the ultimate automated local video editor. It doesn't need to scale to 5,000 cloud users; it needs to run flawlessly on your laptop *right now* and deliver an incredible end-to-end user experience.

Technical Architecture & Constraints

- **No Cloud Required:** Everything should run locally (`localhost`). Local filesystems for storage, local SQLite/JSON for state, and standard Docker Compose or direct local scripts.
- **The Multi-Modal Stack:** You must successfully integrate **Frontend** (UI for upload/preview), **Backend** (API/orchestration), **GenAI** (LLM for creative decisions/scripting), and **ML/Media Processing** (Audio/Video/Image manipulation via OpenCV, FFmpeg, Whisper, or local embedding models).
- **No Pure RAG:** This is *not* a text-search chatbot document processor. It must process live binary media assets (Video, Audio, Images).

Functional Requirements (The Core Build)

1. The Media Ingestion & Analysis Engine (ML Layer)

- **Video Ingestion:** A local API endpoint/UI to drop in an MP4/MOV file (keep test files under 2 minutes for speed).
- **Audio/Visual Analysis:**
 - **Audio:** Extract audio and run a speech-to-text model (e.g., local Whisper or an API like Deepgram/Groq) to get a timestamped transcript.
 - **Visual:** Sample frames using OpenCV or a lightweight local model to detect *visual energy* (e.g., high movement, scene cuts, or face detection).
- **Metadata Payloads:** Generate a unified internal timeline data structure tracking where the "exciting" or "talking" parts are.

2. The Creative Director (GenAI Layer)

- **The Highlight Decision:** Pass the transcript and visual metadata timestamps to a LLM (OpenAI, Anthropic, or local Ollama).

- **The Prompting Mandate:** Instruct the LLM to act as a Hollywood video editor. It must return a structured JSON object containing:
 1. The exact timestamps (**start_time** to **end_time**) of the top 3–5 most engaging clips to stitch together.
 2. An AI-generated text script for a short **Intro Narration** and **Outro Summary** based on the video's context.

3. The Local Render Engine (Heavy Lifting Layer)

- **Programmatic Video Editing:** Use a library like **MoviePy**, **FFmpeg**, or **OpenCV** to programmatically slice the original video based on the LLM's JSON instructions and concatenate them into a single highlight reel.
- **AI Voiceover Integration:** Feed the LLM's generated intro/outro scripts into a TTS (Text-to-Speech) engine (e.g., ElevenLabs API, OpenAI TTS, or local Kokoro-82M/Coqui).
- **Audio Ducking/Mixing:** Overlay the generated AI voiceover onto the start and end of the edited highlight video, gracefully ducking the original video audio.

4. The Interactive Studio (Frontend Layer)

- **The Dashboard:** A clean UI (Next.js, React, Gradio, or Streamlit) where a user can:
 - Upload the raw local video.
 - Watch a live state indicator (e.g., "Analyzing Audio...", "Consulting AI Director...", "Rendering Clips...").
 - **The Finished Product:** A video player side-by-side with the original, showing the newly rendered AI Highlight Reel, the generated AI script, and clickable interactive timestamps.

🕒 6-Hour Milestone Strategy (How to Win)

Because you are using AI coding assistants, **do not build sequentially**. Build vertically. Get a 10-second video flowing through the whole system first, then make it beautiful.

- **Hour 1: The Blueprint & Skeletal API**
 - Spin up the frontend UI and backend server. Establish local file uploads.
 - *Vibe Coding Tip:* Prompt your AI assistant to generate a boilerplate frontend-backend bridge with drag-and-drop file handling instantly.
- **Hour 2: The Analyzers (Whisper & OpenCV)**
 - Extract audio from video using FFmpeg/MoviePy. Pipe it to Whisper to get text timestamps.
 - Verify you can print out timestamps of who said what, when.
- **Hour 3: The GenAI Brain (The JSON Bridge)**
 - Write the LLM prompt. Force the LLM to return strict JSON arrays specifying the cuts.
 - Hook the output up to your backend so it parses smoothly.
- **Hour 4: The Render Rig (FFmpeg / MoviePy)**

- Take the LLM's JSON and slice the actual video.
- Generate the TTS audio file for the narration and overlay it. *Ensure this runs reliably locally.*
- **Hour 5: The UI Polish & Interactivity**
 - Wire the final output video back to the frontend.
 - Add styling, nice loading animations, and display the LLM's thought process (the transcript + why it chose those clips).
- **Hour 6: Ironing & Demo Prep**
 - Test with 2 different sample videos to ensure no hardcoded bugs break the pipeline.
 - Record a quick 2-minute demo showing the local upload, the console processing, and the output playback.



Judging Criteria

1. **Integration Completeness (40%):** Did you successfully connect UI $\rightarrow$ ML Analysis $\rightarrow$ LLM Logic $\rightarrow$ Programmatic Video Render $\rightarrow$ UI Display?
2. **UX & Interactive Feel (20%):** Does the application feel magical to use locally? Does the frontend communicate what the AI is doing in the background?
3. **Creative GenAI Usage (20%):** How smart/funny/accurate are the AI Director's cut choices and the generated voiceover?
4. **Code Craft with AI (20%):** How well did you leverage vibe coding tools to write clean, modular abstractions (e.g., separating the video-slicing logic from the API routing)?



Deliverables for Final Presentation

- **A running local instance** ready to be demoed directly from your machine.
- **A repository** containing a [README.md](#) with the 2 commands needed to start your application (e.g., `npm run dev` and `python main.py` or `docker-compose up`).